

Real-Debrid API Documentation

Implementation details

- Methods are grouped by namespaces (e.g. "unrestrict", "user").
- Supported HTTP verbs are GET, POST, PUT, and DELETE. If your client does not support all HTTP verbs you can override the verb with *X-HTTP-Verb* HTTP header.
- Unless specified otherwise in the method's documentation, all successful API calls return HTTP code 200 with a JSON object.
- Errors are returned with HTTP code 4XX or 5XX, a JSON object with properties "error" (an error message) and "error_code" (optional, an integer).
- Every string passed to and from the API needs to be UTF-8 encoded. For maximum compatibility, normalize to [Unicode Normalization Form C](#) (NFC) before UTF-8 encoding.
- The API sends ETag headers and supports the *If-None-Match* header.
- Dates are formatted according to the Javascript method [date.toJSON](#).
- Unless specified otherwise, all API methods require authentication.
- The API is limited to **250 requests per minute**, all refused requests will return HTTP 429 error and will count in the limit (bruteforcing will leave you blocked for undefined amount of time)

API methods

The Base URL of the Rest API is:

```
https://api.real-debrid.com/rest/1.0/
```

GET	/disable_access_token	Disable current access token
GET	/time	Get server time
GET	/time/iso	Get server time in ISO

/user

GET	/user	Get current user info
-----	-------	-----------------------

/unrestrict

POST	/unrestrict/check	Check a link
POST	/unrestrict/link	Unrestrict a link

POST	/unrestrict/folder	Unrestrict a folder link
PUT	/unrestrict/containerFile	Decrypt container file
POST	/unrestrict/containerLink	Decrypt container file from link

/traffic

GET	/traffic	Traffic informations for limited hosters
GET	/traffic/details	Traffic details on used hosters

/streaming

GET	/streaming/transcode/{id}	Get transcoding links for given file
GET	/streaming/mediaInfos/{id}	Get media informations for given file

/downloads

GET	/downloads	Get user downloads list
DELETE	/downloads/delete/{id}	Delete a link from downloads list

/torrents

GET	/torrents	Get user torrents list
GET	/torrents/info/{id}	Get infos on torrent
GET	/torrents/activeCount	Get currently active torrents number
GET	/torrents/availableHosts	Get available hosts
PUT	/torrents/addTorrent	Add torrent file
POST	/torrents/addMagnet	Add magnet link
POST	/torrents/selectFiles/{id}	Select files of a torrent
DELETE	/torrents/delete/{id}	Delete a torrent from torrents list

/hosts

GET	/hosts	Get supported hosts
GET	/hosts/status	Get status of hosters
GET	/hosts/regex	Get all supported regex.
GET	/hosts/regexFolder	Get all supported regex for folder links.
GET	/hosts/domains	Get all supported domains.

/settings

GET	/settings	Get current user settings
POST	/settings/update	Update a user setting
POST	/settings/convertPoints	Convert fidelity points
POST	/settings/changePassword	Send verification email to change the password
PUT	/settings/avatarFile	Upload avatar image
DELETE	/settings/avatarDelete	Reset user avatar

/support

Example calls

Here are some example calls, using [cURL](#):

Getting user informations:

Show/hide example

Authentication

Calls that require authentication expect an HTTP header *Authorization* bearing a token, using the following format:

```
Authorization: Bearer your_api_token
```

If you can not send an *Authorization* HTTP header you can also send your token as a parameter in REST API URLs, the parameter is called *auth_token*:

```
/rest/1.0/method?auth_token=your_api_token
```

This token can either be your [private API token](#), or a token obtained using OAuth2's three-legged authentication.

Warning: Never **ever** use your private API token for public applications, it is insecure and gives access to all methods.

Authentication for applications

First, you must create an app in your control panel.

Once you have created an app, you are provided a **client_id** and **client_secret** that you will use for the authentication process.

Opensource Apps

You can use this client ID on opensource apps if you don't need custom scopes or name:

```
X245A4XAIBGVM
```

This app is allowed on following scopes: unrestrict, torrents, downloads, user

This client ID can have stricter limits than service limits due to poorly designed apps using it.

Which authentication process should you use?

- If your application is a website: [three-legged OAuth2](#).
- If your application is a mobile app: [OAuth2 for devices](#).
- If your application is an opensource app or a script: [OAuth2 for opensource apps](#).

The Base URL of the OAuth2 API is:

```
https://api.real-debrid.com/oauth/v2/
```

Workflow for websites or client applications

This authentication process uses three-legged OAuth2.

The following URLs are used in this process:

- **authorize** endpoint: */auth*
- **token** endpoint: */token*

Note: if your application is not a website, you will have to make the user do these steps in a web view (e.g. `UIWebView` on iOS, `WebView` on Android...).

Full workflow

1. Your application redirects the user to Online.net's **authorize** endpoint, with the following query string parameters:

- `client_id`: your app's `client_id`
- `redirect_uri`: one of your application's redirect URLs (must be [url encoded](#) :)
- `response_type`: use the value "code"
- `state`: an arbitrary string that will be returned to your application, to help you check against CSRF

Example URL for authorization:

```
https://api.real-debrid.com/oauth/v2/auth?client_id=ABCDEFGHJKLM&redirect_uri=https%3A%2F%2Fexample.com&response_type=code&state=iloverd
```

2. The user chooses to authorize your application.

3. The user gets redirected to the URL you specified using the parameter `redirect_uri`, with the following query string parameters:

- `code`: the code that you will use to get a token
- `state`: the same value that you sent earlier

4. Using the value of `code`, your application makes a direct POST request (not in the user's browser) to the **token** endpoint, with the following parameters:

- `client_id`
- `client_secret`
- `code`: the value that you received earlier
- `redirect_uri`: one of your application's redirect URLs
- `grant_type`: use the value "authorization_code"

Example cURL call to obtain an access token:

```
curl -X POST "https://api.real-debrid.com/oauth/v2/token" -d "client_id=ABCDEFGHJKLM&client_secret=abcdefghsecret0123456789&code=ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789&redirect_uri=https://your-app.tld/realdebrid_api&grant_type=authorization_code"
```

5. If everything is correct, the access token is returned as a JSON object with the following properties:

- `access_token`
- `expires_in`: token validity period, in seconds
- `token_type`: "Bearer"
- `refresh_token`: token that only expires when your application rights are revoked by user

6. Your application stores the access token and uses it for the user's subsequent visits.

Your application must also store the refresh token that will be used to get new access tokens once their validity period is expired.

Workflow for mobile apps

This authentication process uses a variant of OAuth2, tailored for mobile devices.

The following URLs are used in this process:

- **device** endpoint: `/device/code`
- **token** endpoint: `/token`

Note: you may have to make the user do some steps in a web view (e.g. UIWebView on iOS, WebView on Android...) if you want to do all these steps from the mobile app.

Full workflow

1. Your application makes a direct request to the **device** endpoint, with the query string parameter `client_id`, and obtains a JSON object with authentication data that will be used for the rest of the process.

Example URL to obtain authentication data:

```
https://api.real-debrid.com/oauth/v2/device/code?client_id=ABCDEFGHijklm
```

Example authentication data:

```
{
  "device_code": "ABCDEFGHijklmnopqrstuvwxyz0123456789",
  "user_code": "ABCDEF0123456",
  "interval": 5,
  "expires_in": 1800,
  "verification_url": "https://real-debrid.com/device"
}
```

2. Your application asks the user to go to the **verification** endpoint (provided by `verification_url`) and to type the code provided by `user_code`.
3. Using the value of `device_code`, every 5 seconds your application starts making direct POST requests to the **token** endpoint, with the following parameters:
 - `client_id`
 - `client_secret`
 - `code`: the value of `device_code`
 - `grant_type`: use the value `"http://oauth.net/grant_type/device/1.0"`

Example cURL call to obtain an access token:

```
curl -X POST "https://api.real-debrid.com/oauth/v2/token" -d "client_id=ABCDEFGHijklm&client_secret=abcdefghsecret0123456789&code=ABCDEFGHijklmnopqrstuvwxyz0123456789&grant_type=http://oauth.net/grant_type/device/1.0"
```

Your application will receive an error message until the user has entered the code and authorized the application.

4. The user enters the code, and then logs in if they aren't logged in yet.
5. The user chooses to authorize your application, and can then close the browser window.
6. Your application's call to the **token** endpoint now returns the access token as a JSON object with the following properties:

- `access_token`
- `expires_in`: token validity period, in seconds
- `token_type`: "Bearer"
- `refresh_token`: token that only expires when your application rights are revoked by user

7. Your application stores the access token and uses it for the user's subsequent visits.

Your application must also store the refresh token that will be used to get new access tokens once their validity period is expired.

Workflow for opensource apps

This authentication process is similar to OAuth2 for mobile devices, with the difference that opensource apps or scripts can not be shipped with a **client_secret** (since it's meant to remain secret).

The principle here is to get a new set of `client_id` and `client_secret` that are bound to the user. You may reuse these credentials by using [OAuth2 for mobile devices](#).

Warning: You should not redistribute the credentials. Usage with another account will display the UID of the user who obtained the credentials. E.g. instead of displaying "The most fabulous app" it will display "The most fabulous app (UID: 000)".

The following URLs are used in this process:

- **device** endpoint: `/device/code`
- **credentials** endpoint: `/device/credentials`
- **token** endpoint: `/token`

Full workflow

1. Your application makes a direct request to the **device** endpoint, with the query string parameters `client_id` and `new_credentials=yes`, and obtains a JSON object with authentication data that will be used for the rest of the process.

Example URL to obtain authentication data:

```
https://api.real-debrid.com/oauth/v2/device/code?client_id=ABCDEFGHijklm&new_credentials=yes
```

Example authentication data:

```
{
  "device_code": "ABCDEFGHijklmnopqrstuvwxyz0123456789",
  "user_code": "ABCDEF0123456",
  "interval": 5,
  "expires_in": 1800,
  "verification_url": "https://real-debrid.com/device"
}
```

2. Your application asks the user to go to the **verification** endpoint (provided by `verification_url`) and to type the code provided by `user_code`.
3. Using the value of `device_code`, every 5 seconds your application starts making direct requests to the **credentials** endpoint, with the following query string parameters:

- `client_id`
- `code` : the value of `device_code`

Your application will receive an error message until the user has entered the code and authorized the application.

- The user enters the code, and then logs in if they aren't logged in yet.
- The user chooses to authorize your application, and can then close the browser window.
- Your application's call to the **credentials** endpoint now returns a JSON object with the following properties:
 - `client_id` : a new `client_id` that is bound to the user
 - `client_secret`
 Your application stores these values and will use them for later requests.
- Using the value of `device_code`, your application makes a direct POST request to the **token** endpoint, with the following parameters:
 - `client_id` : the value of `client_id` provided by the call to the **credentials** endpoint
 - `client_secret` : the value of `client_secret` provided by the call to the **credentials** endpoint
 - `code` : the value of `device_code`
 - `grant_type` : use the value "http://oauth.net/grant_type/device/1.0"

The answer will be a JSON object with the following properties:

- `access_token`
- `expires_in` : token validity period, in seconds
- `token_type` : "Bearer"
- `refresh_token` : token that only expires when your application rights are revoked by user

Example cURL call to obtain an access token:

```
curl -X POST "https://api.real-debrid.com/oauth/v2/token" -d "client_id=ABCDEFGHIIJKLM&client_secret=abcdefghsecret0123456789&code=ABCDEFGHIIKLMNOPQRSTUVWXYZ0123456789&grant_type=http://oauth.net/grant_type/device/1.0"
```

- Your application stores the access token and uses it for the user's subsequent visits.
Your application must also store the refresh token that will be used to get new access tokens once their validity period is expired.

Workflow for old apps

Warning: This workflow requires a special authorization on your `client_id` from the webmaster.

The following URLs are used in this process:

- **token** endpoint: `/token`

Full workflow

- Your application makes a direct POST request to the **token** endpoint, with the following parameters:
 - `client_id`

- `username` : User login
 - `password` : User password
 - `grant_type` : use the value "password"
-

Testing Two-Factor Process

For testing purposes only, you can force the server to give you the two factor error by sending:

- `force_twofactor` : true

This will return the two factor validation URL:

- `verification_url` : The URL you should redirect the user to.
- `twofactor_code`
- `error` : "twofactor_auth_needed"
- `error_code` : 11

Workflow if you use a WebView / Popup

Open a WebView / Popup with the value of *verification_url*

Using the value of *twofactor_code*, your application makes a direct POST request (not in the user's browser) to the **token** endpoint, with the following parameters:

- `client_id`
- `code` : the value that you received earlier
- `grant_type` : use the value "twofactor"

Example cURL call to obtain an access token:

```
curl -X POST "https://api.real-debrid.com/oauth/v2/token" -d "client_id=ABCDEFGHJKLMN&code=ABCDEFGHJKLMNPQRSTUVWXYZ0123456789&grant_type=twofactor"
```

You will get a 403 HTTP code until the user inputs the correct security code on *verification_url*.

Workflow if you want to handle the security code validation process

The SMS or email is not sent until you make a request to the **token** endpoint, with the following parameters:

- `client_id`
- `code` : the value that you received earlier
- `grant_type` : use the value "twofactor"
- `send` : true

On success, you will get a 204 HTTP code, if the limit is reached then it will be a 403 HTTP code.

To validate the security code the user gives you, make a request to the **token** endpoint, with the following parameters:

- `client_id`
- `code` : the value that you received earlier
- `grant_type` : use the value "twofactor"
- `response` : use the value the user inputs

On error, you will get a 400 HTTP code, if the number of attempts is reached then you will get a 403 HTTP code.

On success, the answer will be a JSON object with the following properties:

- `access_token`
- `expires_in`: token validity period, in seconds
- `token_type`: "Bearer"
- `refresh_token`

Important: You must **NOT** save any login details, only keep *refresh_token* as the « password ».

Example cURL call to obtain an access token:

```
curl -X POST "https://api.real-debrid.com/oauth/v2/token" -d "client_id=ABCDEFGHijklm&username=abcdefgsecret0123456789&password=abcdefgsecret0123456789&grant_type=password"
```

Get a new access token from a refresh token

The following URLs are used in this process:

- **token** endpoint: */token*

Full workflow

1. Using the value of *refresh_token* your application saved earlier, your application makes a direct POST request to the **token** endpoint, with the following parameters:

- `client_id`
- `client_secret`
- `code`: the value of `refresh_token`
- `grant_type`: use the value "http://oauth.net/grant_type/device/1.0"

The answer will be a JSON object with the following properties:

- `access_token`
- `expires_in`: token validity period, in seconds
- `token_type`: "Bearer"
- `refresh_token`

Example cURL call to obtain an access token:

```
curl -X POST "https://api.real-debrid.com/oauth/v2/token" -d "client_id=ABCDEFGHijklm&client_secret=abcdefgsecret0123456789&code=ABCDEFGHIJklmnopqrstuvwxyz0123456789&grant_type=http://oauth.net/grant_type/device/1.0"
```

List of numeric error codes

In addition to the HTTP error code, errors come with a message (*error* parameter) and a numeric code (*error_code* parameter). The error message is meant to be human-readable, while the numeric codes should be used by your application.

-1	Internal error
----	----------------

1	Missing parameter
2	Bad parameter value
3	Unknown method
4	Method not allowed
5	Slow down
6	Ressource unreachable
7	Resource not found
8	Bad token
9	Permission denied
10	Two-Factor authentication needed
11	Two-Factor authentication pending
12	Invalid login
13	Invalid password
14	Account locked
15	Account not activated
16	Unsupported hoster
17	Hoster in maintenance
18	Hoster limit reached
19	Hoster temporarily unavailable
20	Hoster not available for free users
21	Too many active downloads
22	IP Address not allowed
23	Traffic exhausted

24	File unavailable
25	Service unavailable
26	Upload too big
27	Upload error
28	File not allowed
29	Torrent too big
30	Torrent file invalid
31	Action already done
32	Image resolution error
33	Torrent already active
34	Too many requests
35	Infringing file
36	Fair Usage Limit
37	Disabled endpoint